Group Project Task Sheet

CPT204 AY2526

Introduction

This group project is the final coursework component of the course this semester, which accounts for **40%** of your final mark. You will form a **team of two** with your classmates and apply object-oriented principles and advanced data structures, as learned throughout the semester, to solve the tasks specified in this document.

You will be required to write a report and create a video presentation to demonstrate your understanding of:

1. Object-oriented concepts and software design;
2. Problem-solving techniques using sorting and graph algorithms;
3. AI-assisted team cooperation and project management;
4. Equality, diversity, and inclusion (EDI) principles;
5. Planning, reflection, and self-learning in object-oriented programming.

You are required to submit the following files (see Submission Requirement Section for more details):

- Java code in a ZIP file
- A Word report
- An MP4 video
- A PowerPoint (PPT) presentation used in the video

Stage	Date
Task sheet release	Week 9, Monday, 27th of April, 9:00 am (UTC+8)
Submission	Week 12, Sunday, 24th of May, 11:59 pm (UTC+8)
Submission cut-off	Week 13, Friday, 29th of May, 11:59 pm (UTC+8) *There will be 5% lateness penalty per working day, up to a maximum of 5 working days.* *(Monday-Friday, Week 13). No submissions are accepted after 11:59 pm (UTC+8), 29th of May.*

Project Brief

Scenario

A city plans to carry out a large-scale infrastructure inspection task. Three different information sources each provide a candidate list of locations that may require inspection. Since these datasets come from different systems and processing workflows, they have different properties (e.g., ordered or unordered). Your program is required to process these datasets, identify important inspection targets, and then compute routes for visiting them efficiently.

Overview

In this coursework, you are required to **design and implement an object-oriented Java application** for an **Urban Infrastructure Inspection System**, integrating appropriate data structures, sorting algorithms, graph algorithms, and object-oriented design principles.

The coursework consists of five tasks:

- **Task A** requires you to analyze the behaviour and performance of three sorting algorithms on the three given candidate datasets, and to select the top 10 highest-priority locations from each dataset.
- **Task B** requires you to use graph algorithms to compute route or path results for visiting the selected inspection targets (from Task A) on a weighted graph.
- **Task C** requires you to explain and justify the design of your overall application, with a particular focus on data structures, class design, and object-oriented programming principles.
- **Task D** requires you to reflect on your project process, including AI-assisted planning, teamwork, and equality, diversity, and inclusion considerations.
- **Task E** requires you to present your project in the form of a PPT and a video explanation.

Provided Data

You are given the following three candidate datasets:

- candidates_A.csv
- candidates_B.csv
- candidates_C.csv

Each dataset contains **1000 candidate locations**, with row containing:

- location_id (unique id for locations)
- priority_score (the priority for inspection: the higher the score, the higher the priority)

You are also given one undirected weighted graph edge file:

- paths.csv

Each row in paths.csv represents a weighted edge, containing:

- from_location (starting node of an edge)
- to_location (ending node of an edge)
- weight (distance of an edge)

Task A - Algorithm Evaluation: Sorting

In this task, you need to critically evaluate the performance of different sorting algorithms for the candidate-location selection process in the Urban Infrastructure Inspection System.

Using the **three candidate datasets described above**, you must evaluate the following sorting algorithms taught in class:

- Bubble Sort
- Quick Sort
- Merge Sort

All sorting algorithms must use the **same ranking rule**:

- Sort by priority_score in **descending** order.
- If two rows have the same priority_score, sort by location_id in **ascending** order.

For each dataset, your program must:

1. read the dataset from the CSV file;
2. sort the data using the required ranking rule;
3. measure and compare the running time of the three sorting algorithms;
4. identify the **top 10 highest-priority locations** from that dataset.

After all three datasets have been processed, you should obtain:

- 10 selected locations from Dataset A;
- 10 selected locations from Dataset B;
- 10 selected locations from Dataset C.

In total, this gives **30 selected inspection targets**.

Required Output

In the section of the report, you need to provide a table summarizing the sorting results and timing performance. You may use **System.nanoTime()** or **System.currentTimeMillis()** to calculate the sorting time. To reduce measurement noise, you may run each sorting algorithm multiple times (e.g., 3 times) and report the average runtime.

A suggested format is shown below:

Dataset	Bubble (ns/ms)	Quick (ns/ms)	Merge (ns/ms)	Top 10 Selected Locations
Dataset A				
Dataset B				
Dataset C				

Required analysis

In addition, you need to analyse and compare the three algorithms' performance by considering the following questions in your report:

- How does the initial order of the input data affect the performance of Bubble Sort, Quick Sort, and Merge Sort on the three datasets?
- Which of the three sorting algorithms performs best on each dataset? Explain your answer with reference to the data characteristics.
- Which sorting algorithm behaves most consistently across the three datasets? Why?
- If you were allowed to use only one sorting algorithm in the final system, would you choose the one with the best average runtime, the most stable behaviour, or the simplest implementation? Why?
- If the number of candidate locations became significantly larger, which sorting algorithm would be the most suitable for this system? Why?
- If both runtime efficiency and memory usage are considered, how would your algorithm choice be affected?

Task B - Algorithm Evaluation: Graph

After completing Task A, you will have obtained **30 selected inspection targets** in total, that is, **10 locations from each of the three candidate datasets**.

In Task B, you are required to use these selected targets to identify several **shortest paths** on the weighted graph provided in **paths.csv**, and to evaluate the graph algorithm used in your application.

Recall that **paths.csv** contains the edge information of an undirected weighted graph. Each row represents a weighted connection between two locations, and the connection can be travelled in both directions. Each row contains:

- from_location
- to_location
- weight

Note: The weighted graph in paths.csv defines the **complete infrastructure network**. The 30 selected targets obtained in Task A should be treated as important nodes **within this graph**, rather than as the only nodes in the graph.

Required Output

In the report, you must provide the results for the following four shortest-path query cases:

- **Case 1:** from the **1st selected location in Dataset A** to **itself**.
- **Case 2:** from the **1st selected location in Dataset A** to the **10th selected location in Dataset A**.
- **Case 3:** from the **1st selected location in Dataset A** to the **1st selected location in Dataset B**, **via the 5th selected location in Dataset B**.
- **Case 4:** find the shortest path from the **1st selected location in Dataset A** to the **1st selected location in Dataset C**, such that the path must first pass through the **5th selected location in Dataset B** and then through **the 5th selected location in Dataset C**, in this order.

For each case, you must provide:

- the **starting node (e.g., L0001)**;
- the **destination node (e.g., L0010)**;
- the **shortest path found by your program (e.g., L0001 -> L0002 -> …)**;
- the **total path cost (distance)**.

You can either report the outputs in a table or include relevant console outputs as screenshots in your report to show that the program runs correctly.

Required analysis

Besides reporting the outputs, you should also evaluate the implementation of your graph algorithm by considering the following questions:

- What graph algorithm did you use in your program, and why is it suitable for the weighted graph in this coursework?
- How is your chosen graph algorithm implemented in the program, and what are its time and space complexities (in Big-O notation)?
- If each shortest-path query, such as those considered in the cases 1-4 above, is solved optimally, does that mean the whole inspection-planning problem is solved optimally? Why or why not?
- What alternative graph algorithms could be considered if the graph were unweighted? How might they compare with your current choice?

- What alternative graph algorithms or route-planning approaches might become more suitable if the graph became much larger, or if node coordinates were available? Discuss their possible advantages and limitations.

Task C - Design of the Overall Application

In this task, you are required to explain and justify the design of the overall Urban Infrastructure Inspection System as a complete object-oriented Java application.

In the report, you must describe and justify the design of your application by considering the following aspects.

Data Structure

- What data structure(s) did you use to represent the three candidate datasets?
- What data structure(s) did you use to represent the weighted graph in paths.csv?
- Why are these data structures suitable for sorting, graph traversal, and shortest-path computation?
- How do these data structures support the efficiency and clarity of your implementation?

Classes and Functions

- What classes did you use in your application?
- What are the responsibilities of these classes?
- What important public or private functions do these classes have?
- How do these classes and functions collaborate to complete the whole inspection workflow?

(You may illustrate using UML diagrams or code screenshots)

Object-Oriented Design

- What object-oriented principles (e.g., encapsulation, inheritance, polymorphism, abstraction) have been applied in your overall Java program solution?
- How are these principles applied specifically? For example, how did you use polymorphism to support different behaviours through a common interface or abstraction?
- Why are these OOP principles important to your program?

(You may illustrate using UML diagrams or code screenshots)

Task D - Project Reflection

The purpose of this task is to encourage you to think beyond the technical implementation of the program and reflect on how the project was planned, developed, and evaluated as a team. Your reflection should show what you learned from the project, how you used available tools and resources, and how you understand broader software-development considerations. In the report, you should discuss the following aspects.

AI-assisted planning and collaboration

- How do you use AI tools (such as JIRA and Trello, , as introduced in the class) to help with the planning and collaboration (e.g., task allocation) in the project?
- What do you think are the advantages and disadvantages of using AI-empowered project management tools?

Equality, Diversity, and Inclusion

- What is your understanding about equality, diversity, and inclusion in the context of this project? Why are these principles important in software design?
- Based on these principles, in what areas can the current project be optimized in the future? (e.g., based on the equality principle, one may think of providing features like text-to-speech for visually impaired users and so on).
- What challenges might arise when applying the improvement above, and how can these challenges be addressed?

Life-long learning and future improvement

- What did this project teach you about life-long learning in software development?
- How did you contribute to the development of advanced software components in this project as a developer in a team?
- What would be your next step if this application were developed further in the future?

Task E - Project Presentation

Finally, you are required to create a video explanation with PPT slides to illustrate your project in a succinct manner with the following requirements.

- Briefly introduce the project (e.g., project purpose, key objectives, and the role of the three candidate datasets and the weighted graph).
- Explain how the sorting algorithms are applied in Task A, including how the top 10 locations are selected from each dataset.
- Explain how the graph algorithm is applied in Task B, including how the shortest-path queries are defined and how the final path results are obtained.
- Explain how object-oriented principles are reflected in the overall application design in Task C.

- Conclude your presentation with a proper reflection regarding planning, collaboration, application of equality, diversity, and inclusion principles, and future development.

Report Requirements

Structure

The purpose of your report is to show how you solved **Tasks A, B, C, and D** in a well-detailed manner. You need to organize your report in the following format:

- Coursework submission cover page
  (This page is **compulsory**. You can download the template from the CPT204 LMO homepage if required.)
- Chapter 1 – Sorting Algorithm (Task A)
  - (You may add sub-chapters)
- Chapter 2 – Graph Algorithm (Task B)
  - (You may add sub-chapters)
- Chapter 3 – Design of the Overall Application (Task C)
  - (You may add sub-chapters)
- Chapter 4 – Project Reflection (Task D)
  - (You may add sub-chapters)
- Chapter 5 – Program Code
  - Include all your code in Chapter 5 as text. Copy and paste each source file content into the report. **The use of screenshots in Chapter 5 should not be accepted.**
- Chapter 6 – Appendix
  - You may use this section to include any supplementary sources (e.g., console outputs, extra sorting results, shortest-path outputs, UML diagrams not included in the main text).
- Chapter 7 – Contribution Form
  - Use the following table to indicate your contribution:

Student ID	Contribution

*Note: The total percentage is **100%**. That is, if you and your teammate contribute equally to this project, both of you should fill **50%** in the table.*

Formatting

Write your report using Word with the following formattings:

- Font: **Calibri**
- Font Size: **12**
- Line Spacing: **1.5**
- Normal Margins

Additional requirements:

- The page limit is a maximum of **20 pages**, excluding the cover page and Chapters 5–7.
- Consider using images, tables, UML diagrams, and screenshots of outputs to improve the readability of your report.

Coding

- You must only use libraries that are covered in CPT204 (including those introduced in class and/or in Liang's textbook, where applicable).
- The use of third-party libraries that are not covered in CPT204 may result in penalties according to module policy.

Presentation Requirements

Create a PPT presentation and video explanation (Task E) satisfying the following requirements:

- Clearly demonstrate your understanding of the project in terms of algorithm implementation, application design, analysis, and project reflection in a succinct manner (see Task E description for more details).
- You can use any template for your PPT, not limited to the XJTLU standard theme.
- The video must not exceed **8 minutes** in length.
- The video must be recorded **jointly** by both students, and each student should present for roughly an **equal amount of time** (for example, each student may present about half of the content).
- You must use your own voice for narration and appear on camera (i.e., both of the two students' faces in the video, both one-at-a-time appearing and appearing simultaneously are acceptable).

Exceeding the time limit, using English audio translation software for narration, or failing to show your faces will result in a score of **0 for Task E**.

Submit to the LearningMall:

- A video file in MP4 format;
- The PPT file presented in the video.

Submission Requirement

Submit the following files in accordance with the requirements above. Four separate submission portals (links) will be set up for each file on LMO:

- A ZIP file containing **ALL** your code files;
- A Word file for the report;
- An MP4 file for the video recording;
- A PPT file that is used in the video recording.

Marking Metrics

Metrics for Project Report (Tasks A–D, 80/100)

Task	Weight	Criteria
Chapter 1 - Task A	20	**Correct implementation of sorting algorithms**: The report should clearly present the implementation and comparison of Bubble Sort, Quick Sort, and Merge Sort on the three given datasets. **Correct target selection**: The report should correctly identify the top 10 locations from each dataset according to the required ranking rule. **Analysis of dataset properties**: The report should explain how the different properties (e.g., ordered or unordered) of the three datasets influence sorting behaviour and performance. **Justification of algorithm choice**: The report should critically explain which sorting algorithm would be most suitable for the application under different considerations such as runtime stability, scalability, and memory usage.
Chapter 2 - Task B	20	**Correct construction of the weighted graph:** The program should correctly read paths.csv, interpret the edge data properly, and build an appropriate weighted graph representation for path computation. **Correct application of the graph algorithm:** The program should correctly apply the chosen graph algorithm to the required shortest-path cases, including the handling of direct queries and constrained queries with waypoint(s), where applicable. **Correct query (case) handling and output**: The report should clearly present the required graph cases, including

		direct shortest path, self-to-itself case, and constrained queries with waypoint(s), where applicable. The report should clearly show the starting node, destination node, required waypoint(s), shortest path result, and total path cost for each case. **Algorithm analysis**: The report should justify the chosen graph algorithm, explain its complexity, and critically discuss shortest-path correctness, local vs global optimality, and possible alternative graph approaches under different conditions.
Chapter 3 - Task C	20	**Clarity and correctness of the overall application design**: The report should explain how Tasks A and B are integrated into one coherent Java application. **Selection and justification of data structures**: The report should clearly describe the data structures used for candidate datasets and graph representation, and explain why they are suitable. **Classes and functions**: The report should explain the major classes, methods, and how they collaborate in the workflow. **Object-oriented design**: The report should meaningfully discuss how object-oriented principles, e.g., encapsulation, inheritance, polymorphism, and abstraction, are reflected in the actual implementation, and why these principles are important to the program.
Chapter 4 - Task D	20	**AI-assisted planning and collaboration**: The report should clearly explain how AI-assisted tools supported planning, task allocation, collaboration, or project management, and critically evaluate their advantages and disadvantages. **EDI awareness**: The report should show understanding of equality, diversity, and inclusion in the context of the project, and critically discuss how these principles could influence future improvement of the application. **Life-long learning and team role**: The report should reflect on what the project taught the student about life-long learning in software development, how they contributed to the development of advanced software components, and what role they played in the project. **Depth of reflection**: The discussion should be thoughtful, specific, and clearly based on the actual project experience.

Metrics for Code and Project Presentation (20/100)

Task	Weight	Criteria
Chapter 5 - Code	10	**Code clarity, organization, and readability**: Includes appropriate variable naming, comments, documentation, and code structure. The code should correctly support the required sorting workflow, graph query workflow, and overall application logic. **Design quality**: The code should reflect a coherent object-oriented design rather than a collection of unrelated functions.
PPT + Video	10	**Logical flow of presentation**: The slides and spoken explanation should cover the project in a clear and structured way. **Coverage of key tasks**: The presentation should explain sorting, graph algorithms, overall application design, and project reflection appropriately. **Clarity and fluency**: Presenters should speak clearly and explain the project accurately. **Effectiveness of communication**: The PPT should make good use of visual support to improve understanding.

Academic Integrity

1. For this assignment, you are permitted to make **non-substantive use** of AI. This means that AI can only be employed for minor tasks such as grammatical corrections, formatting adjustments, or generic suggestions (e.g., “improve clarity”). You should maintain a record of unedited versions of your work prior to processing by AI, which you may be required to provide as part of the verification process. If you are unsure on whether a specific use of AI is allowed or not, please consult the module leader. In the appendix section of your report, you should include a brief citation of the tool name and version, and what it was used for, e.g.: [1] DeepSeek V3.1, available at https:www.deepseek.com. Used for grammatical proofreading.
2. In some cases, individual students may be invited to explain parts of their code in person, and if they fail to demonstrate an understanding of the code, no credit will be given for that part.

3. In more severe cases, the suspected violation will be directly reported to the Exam Officer for further investigation and, if confirmed, will be permanently recorded in the offender student's official academic transcript.
4. Please read: https://academicpolicy.xjtlu.edu.cn/article.php?id=138